

TRACK 4: HUMANOID-ROBOT MOTION GENERATION

Predictive Risk-Guided Residual Control for Text-Driven Humanoid Manipulation

1 Topic, Scope, and Objectives

We will study **text-driven tabletop block grasping with a simulated Unitree G1**. Under the instructor-approved scope, we will compare one frozen text-to-motion baseline with the same system augmented by **Predictive Risk + Residual Correction**, using the same frozen SONIC controller. All task data, evaluation, and demonstrations will come from simulation.

Our research question is: **Can a learned representation of where and when execution problems may occur improve selective residual correction beyond a binary gate or a scalar risk score?** Our objectives are to:

- Establish a reproducible ARDY-SONIC grasp-and-lift baseline in simulation.
- Learn small, selective corrections while keeping ARDY and SONIC frozen.
- Isolate prediction, risk structure, and loss contributions; measure grasp success, response time, user corrections, and physical feasibility.

Residual robot control is established [1]; recent work also studies gated body-hand residuals with frozen nominal modules [2]. Our focus is a controlled study of **future-time/body-structured risk conditioning** for text-driven contact tasks. Supervised correction will be used in this project; reinforcement learning is outside the core scope.

2 Selected Model and Simulation System

Model choice: ARDY-G1-RP-25FPS-Horizon8. ARDY supports streaming text and sparse spatial constraints, and its project reports integration with SONIC [3]. We will use the official implementation [4] and G1 checkpoint [5]. Its short chunks suit repeated correction. This selection reflects task fit; its grasp success and hardware throughput must be measured.

MuJoCo environment. We will combine SONIC [6] and its official simulation setup [7] with MuJoCo [8], a free-base G1, table, dynamic block, and articulated hand. Lifting must result from frictional hand contacts. Standing constraints and SONIC provide lower-body stabilization without fixing the base. Robot/hand assets, physics step, actuator gains, and contact settings will be versioned and identical across comparisons. The initial task is single-hand grasping of one block at reachable tabletop poses.

Text grounding and execution. A shared sequencer will parse reaching, directional adjustment, grasping, and lifting commands into phases and wrist goals. Current simulator object poses provide geometric grounding; the core study explicitly uses privileged state. The parser, constraints, and grasp logic will be documented, so language performance is attributed to the complete pipeline. ARDY receives text, pose history, and constraints; camera perception is deferred.

The adapter will use SONIC’s joint-based streaming interface [9]: 29 body-joint positions/velocities in the required order, body orientation, and aligned frame indices. We will pin one matching encoder/decoder/config set, initially the default release. ARDY output will be decoded, mapped, and resampled into nominal references A_t :

$$\text{Text} + \text{goals} \rightarrow \text{ARDY} + \text{adapter} \rightarrow A_t \xrightarrow{\text{Risk} + \text{Residual}} A_t + \Delta A_t \rightarrow \text{SONIC} \rightarrow \text{simulated G1}.$$

Residuals change arm-joint references only; velocities and dependent reference fields are recomputed afterward. Root, legs, and fingers receive no learned offsets. A shared finger controller handles closure/release and contact checks. All systems use identical reference limits, clearance checks, and failure stops. A baseline bypasses the learned modules. Recovering falls through leg corrections is outside this initial controller’s authority.

3 System Architectures

Both architectures share the same frozen models, hand controller, and physics.

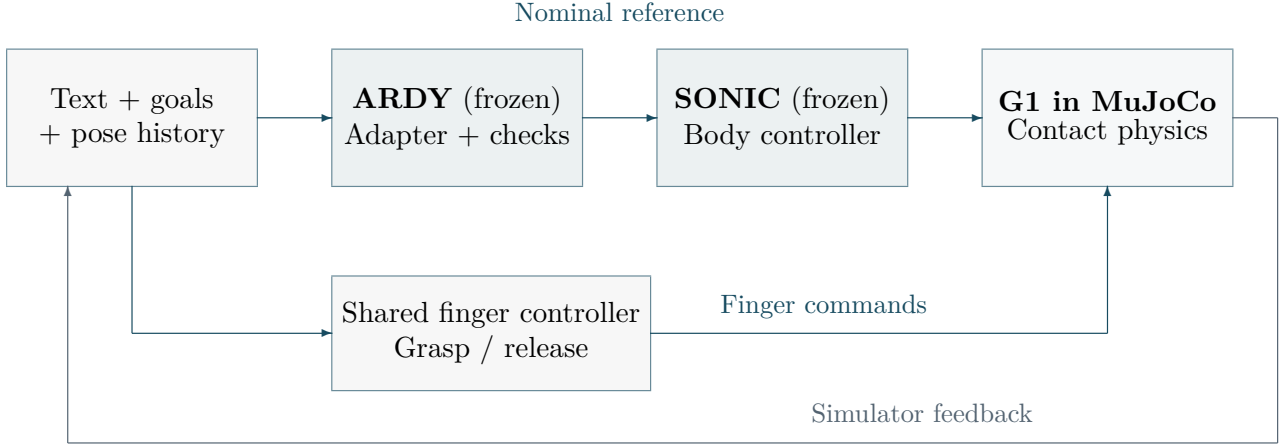


Figure 1: **Baseline (B0): ARDY + SONIC**. Frozen ARDY supplies nominal references to SONIC. The shared task interface controls the fingers and consumes simulator feedback.

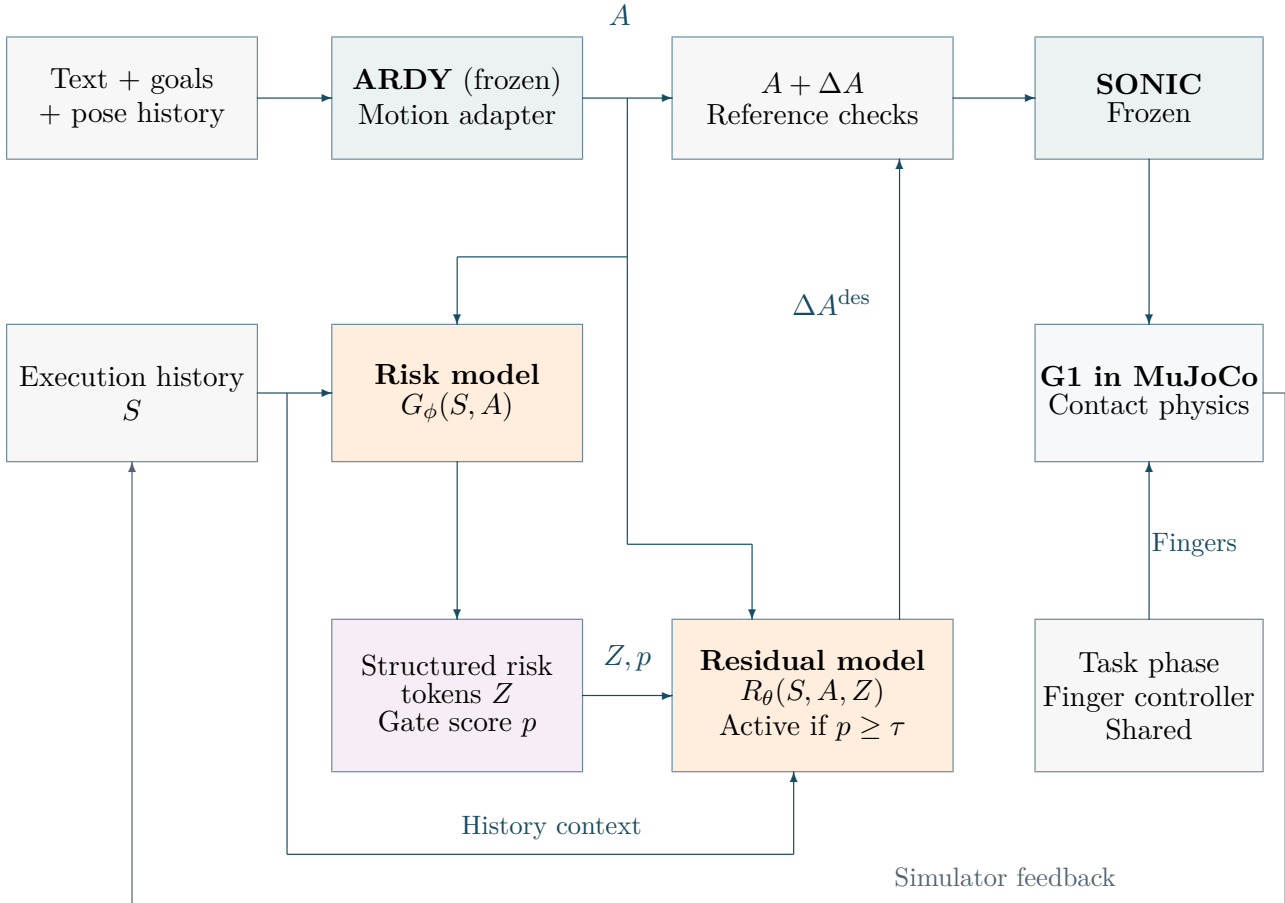


Figure 2: **Proposed (P): ARDY + Risk + Residual + SONIC**. Risk uses history and future nominal motion. Structured tokens condition the bounded residual; low risk requests zero correction. Orange modules are trainable; blue modules are frozen. Reference checks smooth gate transitions; shared hand control and feedback match Figure 1.

4 Predictive Risk and Residual Learning

4.1 Anticipating execution problems

The risk model receives 200–500 ms of execution history and future nominal references, with common task-phase and planned finger-command context:

$$S_t = [s_{t-K+1}, \dots, s_t], \quad A_t = [a_t, \dots, a_{t+H-1}], \quad (Z_t, p_t) = G_\phi(S_t, A_t).$$

State includes joint positions/velocities, base orientation/angular velocity, hand-object pose, contacts, tracking error, and task phase. Current/past observations are inputs; future actual execution is used only for labels. The hand-command context is observed but never changed by the residual.

A compact Transformer (width 256, four layers, eight heads) will produce $Z_t \in \mathbb{R}^{H \times B \times D}$ with body/time positional embeddings: $H = 8$ samples spaced 40 ms apart, $B = 4$ groups (left/right arm-hand, torso, lower body), and $D = 32$. Body/time auxiliary heads supervise tracking error, phase-dependent contact loss, and instability; a pooled head gives p_t :

$$\mathcal{L}_{\text{risk}} = \lambda_e \mathcal{L}_{\text{track}} + \lambda_c \mathcal{L}_{\text{contact}} + \lambda_b \mathcal{L}_{\text{balance}} + \lambda_i \mathcal{L}_{\text{intervene}}.$$

Intervention labels mark threshold violations during nominal future execution; thresholds are validated then fixed. Open-hand phases are not grasp failures. Losses mask unavailable future steps. High risk indicates possible failure, not a guarantee that an arm correction can recover it.

4.2 Bounded correction and distinct supervision

A temporal Transformer reads (S_t, A_t, Z_t) . An arm-only mask M and bounds ϵ restrict the prediction; gating selects a desired correction:

$$\widehat{\Delta A}_t = M \odot \text{clip}(R_\theta(S_t, A_t, Z_t), -\epsilon, \epsilon), \quad \Delta A_t^{\text{des}} = \mathbf{1}[p_t \geq \tau] \widehat{\Delta A}_t.$$

The shared reference checks apply magnitude/rate limits before execution. Low risk stops residual inference and requests zero; any previous offset ramps to zero under the same rate limit. Diagrams use ΔA for the applied offset after this step.

Teacher recoveries supply $\Delta A_t^* = A_t^{\text{expert}} - A_t$. Correction and identity losses use **disjoint** recoverable-perturbed and stable-nominal sets $\mathcal{D}_c$ and $\mathcal{D}_s$:

$$\begin{aligned} \mathcal{L}_{\text{res}} = & \mathbb{E}_{\mathcal{D}_c} \left\| \widehat{\Delta A}_t - \Delta A_t^* \right\|_F^2 + \alpha \mathbb{E}_{\mathcal{D}_s} \left\| \widehat{\Delta A}_t \right\|_F^2 \\ & + \beta \mathbb{E}_{\mathcal{D}_c \cup \mathcal{D}_s} \sum_{h=0}^{H-2} \left\| \widehat{\Delta a}_{t,h+1} - \widehat{\Delta a}_{t,h} \right\|_2^2. \end{aligned}$$

This avoids counting stable zero targets twice. Sampling ratios and loss normalization are fixed across ablations; removing identity supervision now has a distinct meaning. Training uses bounded predictions before the gate.

4.3 Simulation data and runtime

An offline constrained IK teacher will propose arm-only corrections within the same bounds, verified through frozen SONIC and the shared hand controller from the **same perturbed state**. Simulated teleoperation may supplement recoveries. Failed recoveries supervise risk only. Nominal-label branches restore simulator state, controller history, reference buffers, and disturbance seeds; corrected outcomes never substitute for nominal counterfactual labels.

Split parent episodes (including all branches) before window extraction. Train risk first, then residuals with **predicted** tokens and a frozen risk encoder on S4000. The teacher is absent at evaluation. Joint fine-tuning is deferred.

SONIC targets 50 Hz; risk/residual updates target 10–20 Hz. A timestamped buffer covers both risk and SONIC lookahead; references are resampled to each clock. Execute only until the next update before replanning. Underruns trigger a shared hold and are logged. Simulation may run slower than wall time; 25 FPS motion sampling does not establish real-time inference.

5 Primary Comparison and Ablation Experiments

Every configuration uses the **same frozen ARDY and SONIC checkpoints**, scene, task sequencer, hand controller, observations, and reference adapter. The primary comparison is **B0 versus P**; intermediate controls explain where any improvement comes from.

ID	Configuration	Purpose
B0	ARDY + SONIC	Frozen baseline; no learned correction.
B1	Always-on residual	Residual receives history and nominal actions only; tests whether ordinary residual learning is sufficient.
B2	Reactive gate + residual	Current tracking/contact/balance thresholds gate the B1 residual; tests reactive intervention.
I1	Predictive gate only	Predicted risk activates a residual with no risk features; tests anticipation without risk conditioning.
I2	Predictive scalar + residual	Residual also receives the risk probability; tests a single predictive risk score.
I3	Body-wise risk + residual	Residual receives body-wise auxiliary risks; tests spatial localization.
I4	Body-time risk map + residual	Residual receives explicit auxiliary predictions over $H \times B$; tests temporal localization.
P	Structured tokens + residual	Full proposed method: gated residual conditioned on learned $H \times B \times D$ risk features.

5.1 Isolating the risk interface

I1–I4 and P share a frozen risk encoder and gate. B1/B2/I1 reuse one residual checkpoint. Identical body/time slots contain zeros (I1), repeated probability (I2), time-pooled body-wise auxiliary risks (I3), body/time predictions (I4), or learned tokens (P). Threshold-normalized auxiliary channels share feature width, decoder capacity, corrective data, and training budgets. We will report adapter/decoder parameter counts.

The gate function is fixed, but closed-loop states can change activation rates. Compare success–intervention curves at validation-selected thresholds and residual errors on identical held-out windows. Structured features should improve success/locality at comparable intervention effort; this is a hypothesis.

5.2 Component and loss ablations

Each row below changes **one factor** relative to P. Other losses, correction bounds, data, and optimization budgets remain fixed.

Ablation	Controlled change	Question / evidence
Pooled learned tokens	Average body/time tokens and repeat over slots; retrain residual.	Does localization matter beyond a generic learned feature?
No future-action input	Remove future actions from risk only; retrain risk/residual.	Does anticipation improve warning lead time and success?
No execution history	Risk uses current state and future actions; retrain risk/residual.	Does temporal context expose delay and drift?
No auxiliary risk losses	Train risk with intervention loss only; retrain residual.	Do execution-related heads make tokens useful?
No selective gate	Force the trained P gate on; keep both networks fixed.	Does selection reduce unnecessary corrections?
No identity loss	Retrain residual with $\alpha = 0$.	Does removing stable-state supervision increase unnecessary correction?
No smoothness loss	Retrain residual with $\beta = 0$.	Do reference jitter, contacts, or grasp stability worsen?

Hard bounds remain; compare raw and applied offsets because limiting can mask loss effects. Prioritize B0/B1/I2/P and report any unfinished ablations.

6 Evaluation Protocol and Analysis

Prioritized test suite. Core conditions are nominal execution, grasp-target bias, and action latency. The instruction set covers reaching, adjustment, grasping, lifting, paraphrases, and left/right targets. Fixed pilot-calibrated perturbation levels will be published before testing. Mass, friction, pushes, and held-out combinations form a secondary stress suite after core comparisons are complete. Prompt paraphrases and scene seeds are separated from training where applicable.

Success and trial budget. The instructed block’s lowest point must clear the tabletop by 5 cm and remain held for 2s within 30s of simulated time, without a fall or prohibited collision. Fall/clearance thresholds will be fixed in the evaluation config. We plan **20 paired episodes per core condition per configuration**. Match instructions, initial states, perturbations, and ARDY random seeds; closed-loop nominal motions may subsequently diverge. B1, I2, and P target three training seeds; other ablations start with one and are exploratory. The full plan on core conditions is approximately 1,260 episodes (10.5 simulated hours at the timeout limit), before secondary stress tests or runtime overhead.

Metrics and uncertainty. Report success with Wilson 95% intervals, paired success differences with an episode-level bootstrap, and results per training seed. Overlapping windows are not independent trials. Also report command-to-first-motion/completion time, wrist error, slips/falls/collisions, intervention rate, correction norm, locality, and smoothness. Risk metrics include precision–recall, false alarms, and warning lead time on held-out nominal rollouts, avoiding successful interventions being relabeled as false alarms. Latency includes text encoding, generation, communication, risk, and residual inference (median/p95). Simulated task time, wall-clock response, and simulation speed are separate.

Autonomous trials receive no extra user corrections. An interactive subset allows at most two corrections under a fixed rule, excluding planned phase commands from the count. Failed trials and timeouts remain in the denominator; completion times are reported alongside success. Videos will cover language, pose, contact, balance, intervention, and buffering failures.

7 Milestones and Expected Deliverables

Project period: **29 September–15 October 2026**. Integration, data tooling, and model development will run in parallel.

Period	Milestone and acceptance evidence
Sep 29–30	Verify model access and S4000 operators; launch G1/SONIC in MuJoCo. Build data and evaluation tools in parallel.
Oct 1–3	Integrate ARDY, constraints, and hand control; freeze the baseline after a verified contact grasp/lift. Validate teacher recovery targets.
Oct 4–7	Expand perturbation/correction data; train risk and residual models. Validate selective closed-loop correction.
Oct 8–11	Complete B0/B1/I2/P first; then expand interfaces and component ablations. Freeze the completed result set and failures.
Oct 12–14	Freeze results; complete report, slides, repository, and videos. Reproduce key runs and rehearse the presentation.
Oct 15	Final consistency checks and submission of all required deliverables by the course deadline.

By Sep. 30, verify S4000 training operators, text-encoder access, and a working SONIC inference backend [4], [7]. TensorRT compatibility is not assumed; confirm any separate compatible inference host is available. If integration slips, reduce scene variation before expanding experiments. No throughput or successful grasp is claimed before these checks.

Deliverables: the simulation system, checkpoints, ablation results, and a public GitHub repository with setup/data/training/evaluation scripts; a 5–8-page report (excluding references); and a 10-minute presentation with 5-minute Q&A. Videos show commands and successful/failed trials for both systems.

8 Future Directions: VLA Policies in Simulation

After the ARDY study, we propose testing **GR00T N1.7** [10] and $\pi_{0.5}$ [11] with the same risk-guided residual principle, using rendered images, language, and robot state. GR00T documents a G1-SONIC latent-action interface; $\pi_{0.5}$ would require G1-specific adaptation. Each future baseline must first be validated and then frozen. Risk/residual modules would be trained on that policy’s simulated rollouts and evaluated with and without correction under matched perturbations. Correction coordinates and bounds must suit the action interface; adding offsets to latent tokens is not automatically equivalent to correcting joint references. This is future work, and zero-shot weight transfer would be a separate experiment.

9 Team Roles and Responsibilities

Each member owns a technical component and contributes its methods, evidence, and limitations to the report and presentation.

Member	Primary role	Responsibilities
Gu Wentao	Architecture, experiments, and training lead	Design the framework and experiments/ablations; coordinate module interfaces; lead S4000 training, tuning, and checkpoints.
He Yutong	ARDY integration	Integrate text encoding, pose/constraint inputs, and streaming inference; validate the selected checkpoint.
Lu Yinuo	SONIC and adaptation	Implement joint/frame mapping, resampling, and reference updates; validate lower-body tracking and balance.
Shi Yixin	Simulation environment	Build the MuJoCo G1/table/block scene, contact physics, cameras, and reproducible resets.
Sun Yuhan	Predictive risk model	Implement history encoding, auxiliary heads, structured risk tokens, and prediction validation.
Tie Yutong	Residual correction	Implement risk-conditioned residuals, correction bounds, identity/smoothness losses, and model ablations.
Wang Qifa	Data engineering	Build synchronized episode recording, loaders, normalization, split audits, and optional LeRobot export.
Wang Yuxiang	Perturbations, labels, and mathematical reasoning	Generate perturbations, nominal snapshot rollouts, risk labels, and validated corrective targets; analyze mathematical assumptions.
Xia Fangxin	Language grounding and goals	Build the task sequencer, instruction suite, wrist/standing constraints, and IK teacher interface.
Yin Siyuan	Evaluation and statistics	Run paired trials and ablation comparisons; analyze uncertainty, intervention trade-offs, and failure categories.
Zang Ziyi	Simulated hand interaction	Implement finger actuation, contact checks, grasp/lift verification, and hand-model validation.
Zhu Jianyu	Runtime and reproducibility	Implement buffering, timing, launch scripts, video recording, and the reproducible release.

External code, models, and datasets will be acknowledged. LLM assistance was used to draft this proposal; the team will review it under the course policy.

References

- [1] T. Johannink et al., “Residual reinforcement learning for robot control,” in Proc. IEEE Int. Conf. Robot. Autom. (ICRA), Montreal, QC, Canada, 2019, pp. 6023–6029, doi: [10.1109/ICRA.2019.8794127](https://doi.org/10.1109/ICRA.2019.8794127).
- [2] R. Wu, S. Li, L. Zhang, A. Knoll, and Z. Chen, “Gated residual body-hand coordination for whole-body humanoid teleoperation,” 2026, arXiv:2609.18763. [Online]. Available: <https://arxiv.org/abs/2609.18763>
- [3] K. Zhao, M. Petrovich, H. Zhang, T. Wang, S. Tang, and D. Rempe, “ARDY: Autoregressive diffusion with hybrid representation for interactive human motion generation,” ACM Trans. Graph., vol. 45, no. 4, 2026, Art. no. 86, doi: [10.1145/3811284](https://doi.org/10.1145/3811284).
- [4] NVIDIA. “ARDY.” GitHub. Accessed: Sep. 28, 2026. [Online]. Available: <https://github.com/nv-tlabs/ardy>
- [5] NVIDIA. “ARDY-G1-RP-25FPS-Horizon8.” Hugging Face. Accessed: Sep. 28, 2026. [Online]. Available: <https://huggingface.co/nvidia/ARDY-G1-RP-25FPS-Horizon8>
- [6] Z. Luo et al., “SONIC: Supersizing motion tracking for natural humanoid whole-body control,” Sci. Robot., vol. 11, no. 117, 2026, Art. no. eaed4592, doi: [10.1126/scirobotics.aed4592](https://doi.org/10.1126/scirobotics.aed4592).
- [7] NVIDIA. “GR00T-WholeBodyControl.” GitHub. Accessed: Sep. 28, 2026. [Online]. Available: <https://github.com/NVlabs/GR00T-WholeBodyControl>
- [8] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS), Vilamoura, Portugal, 2012, pp. 5026–5033, doi: [10.1109/IROS.2012.6386109](https://doi.org/10.1109/IROS.2012.6386109).
- [9] NVIDIA. “Streaming motion tracking.” GR00T-WholeBodyControl Documentation. Accessed: Sep. 28, 2026. [Online]. Available: <https://nvlabs.github.io/GR00T-WholeBodyControl/tutorials/zmq.html>
- [10] NVIDIA. “Isaac-GR00T N1.7.” GitHub. Accessed: Sep. 28, 2026. [Online]. Available: <https://github.com/NVIDIA/Isaac-GR00T>
- [11] Physical Intelligence. “OpenPI.” GitHub. Accessed: Sep. 28, 2026. [Online]. Available: <https://github.com/Physical-Intelligence/openpi>